糖尿病預測聯邦學習——實作檢查與後續建議

針對目前 Flower 實作的回饋意見與後續行動建議

1. 實作檢查結果

聯邦學習的核心機制實作正確。客戶端、伺服器與模擬腳本之間的串接方式沒有問題，所得到的結果是真實且有意義的。不過有兩個問題應在正式根據這些數字下結論之前先修正，詳見第 5 節。

1.1 實作正確之處

- client.py 正確實作了 Flower 的 NumPyClient 介面（get_parameters、set_parameters、fit、evaluate），並在 simulation.py 中正確地被實例化。
- server.py 中自訂的聚合策略，在每一輪訓練後都能正確地將彙整後的全域模型權重儲存至硬碟，因此最終儲存的模型確實反映了最後完成的那一輪訓練結果。
- 模擬結束後的評估腳本能正確重新載入已儲存的全域權重，並以與專案中其他地方相同的訓練／評估函式，執行全部三種比較情境（僅本地、僅全域、個人化 FL）——這確保了比較的一致性。

1.2 發現的問題

- 伺服器在訓練過程中其實從未將 local_epochs 或 lr 傳送給客戶端，因此無論你們想如何設定，每個客戶端在每一輪都只會默默地使用相同的預設值進行訓練（詳見第 5 節修正一）。
- 程式碼中完全沒有設定隨機種子（random seed），因此每次執行的結果都會有些微差異，這會讓人難以判斷某項改善是真實效果還是隨機誤差（詳見第 5 節修正二）。

2. 認識三種評估情境

每個分區（一個 NHANES 時間區間，視為一個模擬客戶端）都會在自己保留的測試集上，以三種方式進行評估：

2.1 僅本地模型（Local Only）

從頭開始訓練的模型，僅使用該分區自己的訓練資料——完全沒有涉及聯邦學習。這代表如果該群組完全獨立訓練、不與任何其他群組合作，會得到的結果。

2.2 僅全域模型（Global Only）

FL 伺服器在所有訓練輪次結束後所產出的最終模型。由於它是在訓練過程中結合所有分區的更新所建構而成，因此間接學習到了所有群組的資料特性——即便任何一個群組的原始資料從未被分享給其他群組。這個版本是直接以伺服器傳回的狀態進行評估，未經任何本地端調整。

2.3 個人化 FL（Personalized FL）

以最終全域模型的權重為起點，再使用該分區自己的資料額外進行少量訓練，然後評估。其概念是：從所有群組共同學習到的成果出發，再做些微調整，使模型更貼合這個特定群組的資料。

3. 為何「全域模型優於個人化模型」是正常現象

在目前取得的結果中，某些分區的「僅全域模型」表現會優於「個人化 FL」。這是聯邦學習研究中已知且常見的現象，並不代表實作有誤。

原因在於：針對某一分區額外進行的少量本地訓練，可能會讓模型過度配適（overfit）該分區特有的雜訊與特性，尤其當該分區樣本數較少，或類別（糖尿病 vs. 非糖尿病）比例不均衡時更容易發生。這樣的本地微調，反而可能使模型偏離全域模型原本透過整合所有分區資訊所學到、更具一般化能力的模式。這種現象有時被稱為「個人化飄移」（personalization drift）。

該如何呈現在報告中： 這是一項合理且值得探討的發現，而不是需要隱藏的問題。一段好的結果說明可以這樣寫：「個人化微調使 5 個群組中的 3 個表現提升，但使另外 2 個群組的表現略為下降，顯示本地微調較有利於資料量較大或資料分佈較獨特的群組，而對於資料量較小或與其他群組較相似的群組，單純使用全域模型反而效果更好。」這種細緻的分析正是研究所等級報告應該呈現的內容。

4. 什麼樣的改善幅度算是「好」

並沒有單一的通用標準，但對於像這樣的表格型健康預測任務而言，當以準確率、F1 分數或 AUC 的絕對百分點差異，比較 FL 結果（全域或個人化）相對於僅本地模型時，以下範圍可作為合理的參考依據：

改善幅度範圍	解讀
低於 1 個百分點	可能仍落在正常的執行間誤差範圍內，尚不足以構成 FL 確實帶來效益的有力證據
1–3 個百分點	屬於有意義、值得在報告中呈現的改善，尤其若在大多數分區間皆一致出現
3 個百分點以上	屬於值得作為核心發現特別強調的優異結果
各分區表現不一致（有升有降）	並非失敗，而是值得深入分析的現象——探討哪些群組較能受益、哪些不能（如資料量、類別平衡、與其他群組的相似程度）

重要提醒： 僅憑單次執行的結果，並不足以精確信賴這些數字。由於隨機初始化與資料洗牌（shuffling）的影響，不同執行之間出現一、兩個百分點的差異相當常見。在正式回報最終數字之前，應以數個不同的隨機種子重複相同實驗（例如執行 3 到 5 次），並以平均值與標準差呈現結果，而非僅呈現單一數字。

5. 修正現有實作問題所需採取的行動

修正一——讓客戶端訓練設定真正可被調整

客戶端程式碼其實已經寫好接收伺服器傳來的 local_epochs 與 lr 設定，但伺服器端從未真正傳送這些值。請在伺服器策略中加入 on_fit_config_fn，讓這些設定能真正被控制與調整：

```
def fit_config(server_round: int):
    return {"local_epochs": 2, "lr": 0.01}

strategy = SaveModelStrategy(
    ...,
```

```
    on_fit_config_fn=fit_config,
)
```

修正二——設定隨機種子以確保可重現性

在實驗開始時，為 PyTorch、NumPy 以及訓練／測試切分設定固定的隨機種子，使同一組設定每次執行都能得到相同的結果，也讓兩組不同設定之間的差異，能夠被判斷為真實差異，而非隨機變動所致。

修正三——在比較「僅本地」與「FL」之前，先統一訓練量

目前「僅本地模型」只從頭訓練少數幾個訓練週期（epoch），而「全域模型」在所有 FL 訓練輪次累加下，實際上已經歷更多次訓練。這表示 FL 所呈現出的部分改善，有可能只是來自於訓練量較多，而非群組間合作所帶來的效益。在正式回報最終數字之前，應讓「僅本地模型」訓練至相當的總訓練量（或訓練至其自身驗證表現不再提升為止），如此一來比較結果才能真正反映合作所帶來的效益，而非單純的訓練量差異。

修正四——以多組隨機種子執行，並回報平均值與標準差

如第 4 節所述，請以至少 3 組不同的隨機種子重複完整實驗（包含全部三種情境、所有分區），並回報平均結果與離散程度。如此一來，原本單次執行得到的數字，才能成為真正經得起檢驗與論證的結果。

6. 建議優先嘗試的改善方向

在眾多可能的方向中，以下兩項建議作為優先嘗試的項目，因為相對於可能帶來的效益，它們所需投入的心力較低，且能直接針對目前結果中已經觀察到的現象進行改善。

6.1 嘗試以 FedProx 取代原本的 FedAvg

FedProx 是對聚合策略的一項小幅修改，它會限制每個群組在本地訓練時，權重可以偏離全域模型多遠。這項機制特別適用於各群組資料彼此差異較大的情況——而這正是本專案的情境，因為每個分區來自不同的 NHANES 時間區間。此機制透過一個新增的參數來控制（通常稱為 proximal_mu）：數值越高，本地訓練會越貼近全域模型；數值越低，則讓本地訓練有更多自由度。嘗試兩到三種不同數值，並與目前的 FedAvg 設定比較結果，是合理的第一步實驗。

6.2 調整個人化階段的本地微調程度

目前的個人化步驟對所有群組都使用固定的訓練週期數與學習率。既然個人化目前對部分群組有幫助、對另一部分群組反而有害，嘗試少數幾種不同設定（例如減少微調的訓練週期數，或採用較小的學習率），觀察是否能縮小「個人化模型表現不如全域模型」的那些群組之間的落差，會是一個直接且有針對性的改善方式，也與第 3 節中「個人化飄移」的解釋緊密呼應。